

Bases de Datos

Clase 13: Almacenamiento

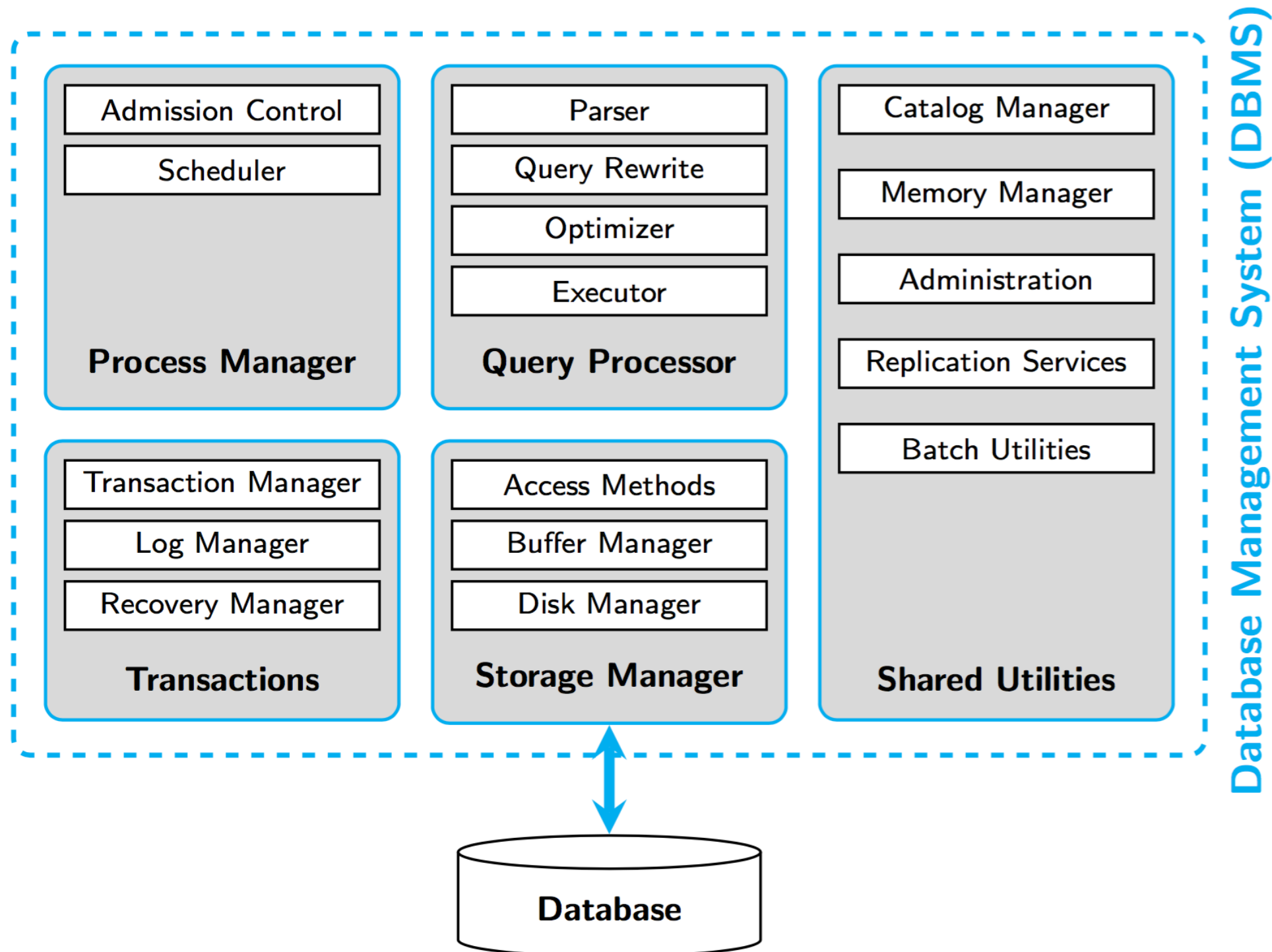
Hasta ahora

Usamos los DBMS como una caja negra: hacemos una consulta y el sistema se encarga

Pero, ¿cómo funciona realmente el sistema?

¿Cómo se evalúa una consulta?

Arquitectura de un DBMS



Ciclo de vida de una consulta

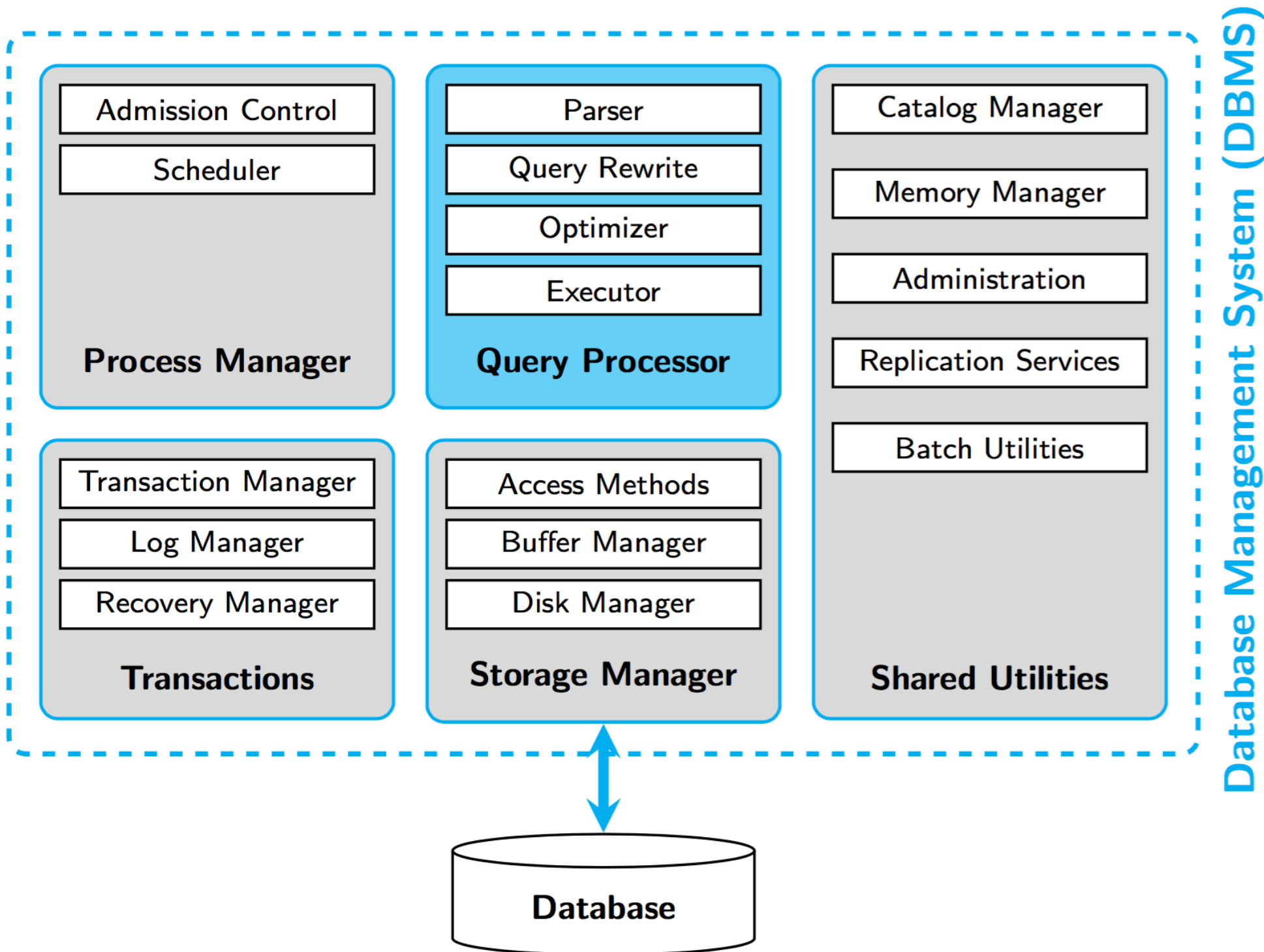
Supongamos la consulta:

```
SELECT pName, mStadium, goals
```

```
FROM Players P, Matches M,  
Players_Matches as PM
```

```
WHERE P.id = PM.id AND PM.mid = M.mid  
AND P.pYear >= 1985
```

Query Processor



Paso 1

Parser

Valida la consulta en base a la sintaxis

Convierte la consulta a álgebra relacional

Optimiza la consulta de forma básica (restricciones numéricas triviales)

Paso 2

Reescritura

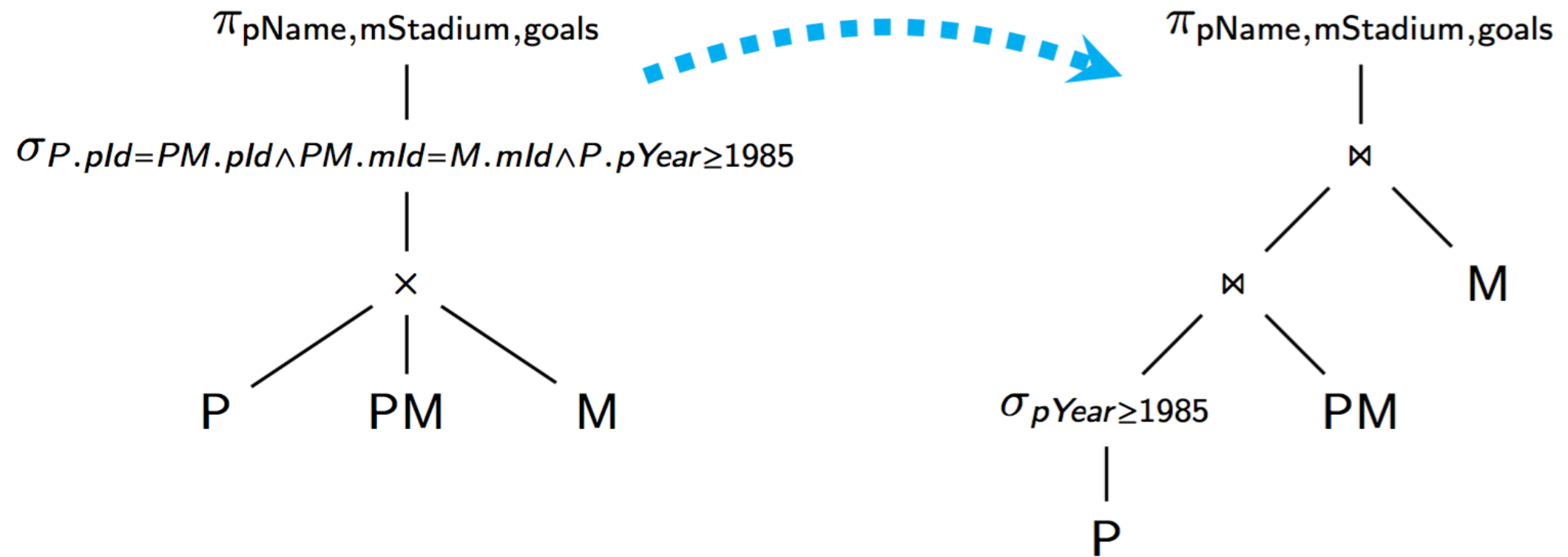
“Desanidación” de la consulta

Reescritura de la consulta aplicando reglas del álgebra relacional

Retorna el plan lógico de la consulta

Paso 2

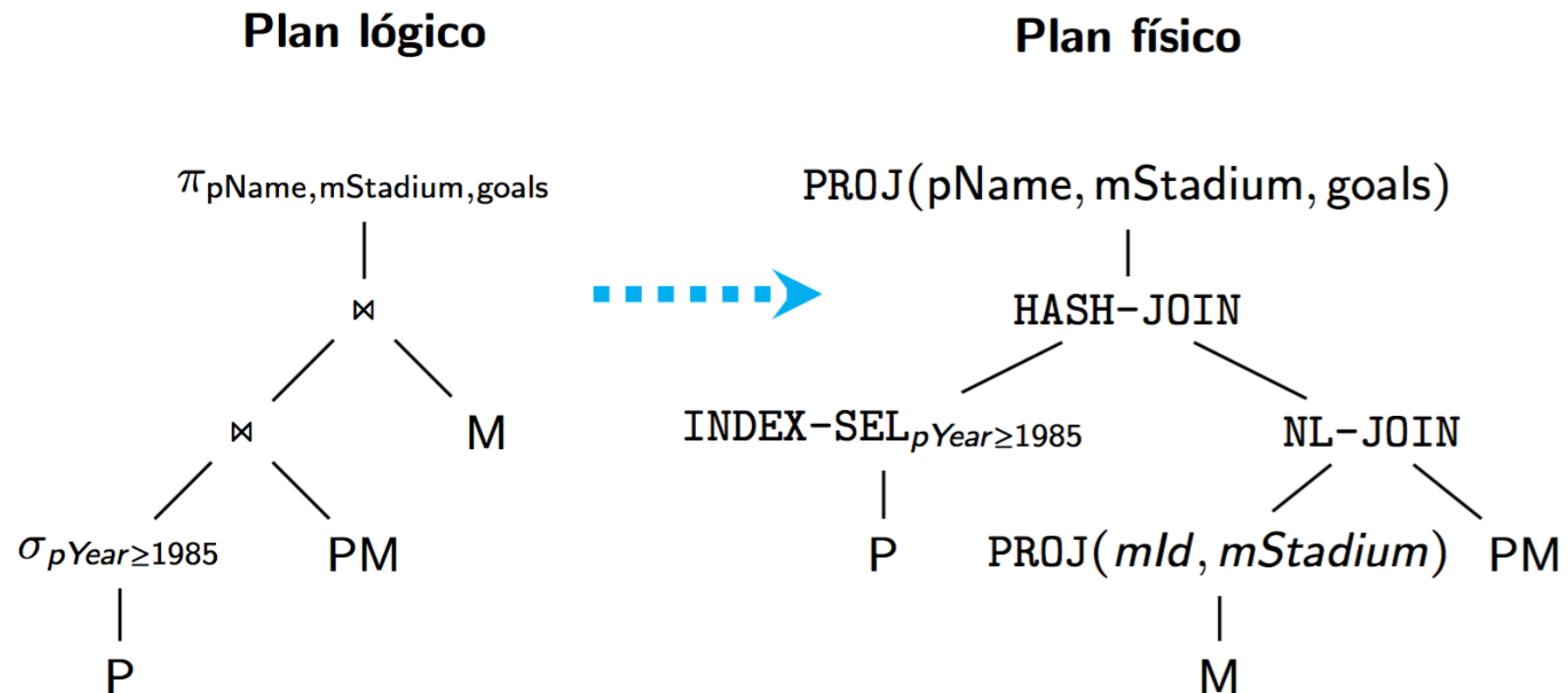
Reescritura



Paso 3

Optimizador

Se encuentra el plan físico más eficiente para la consulta



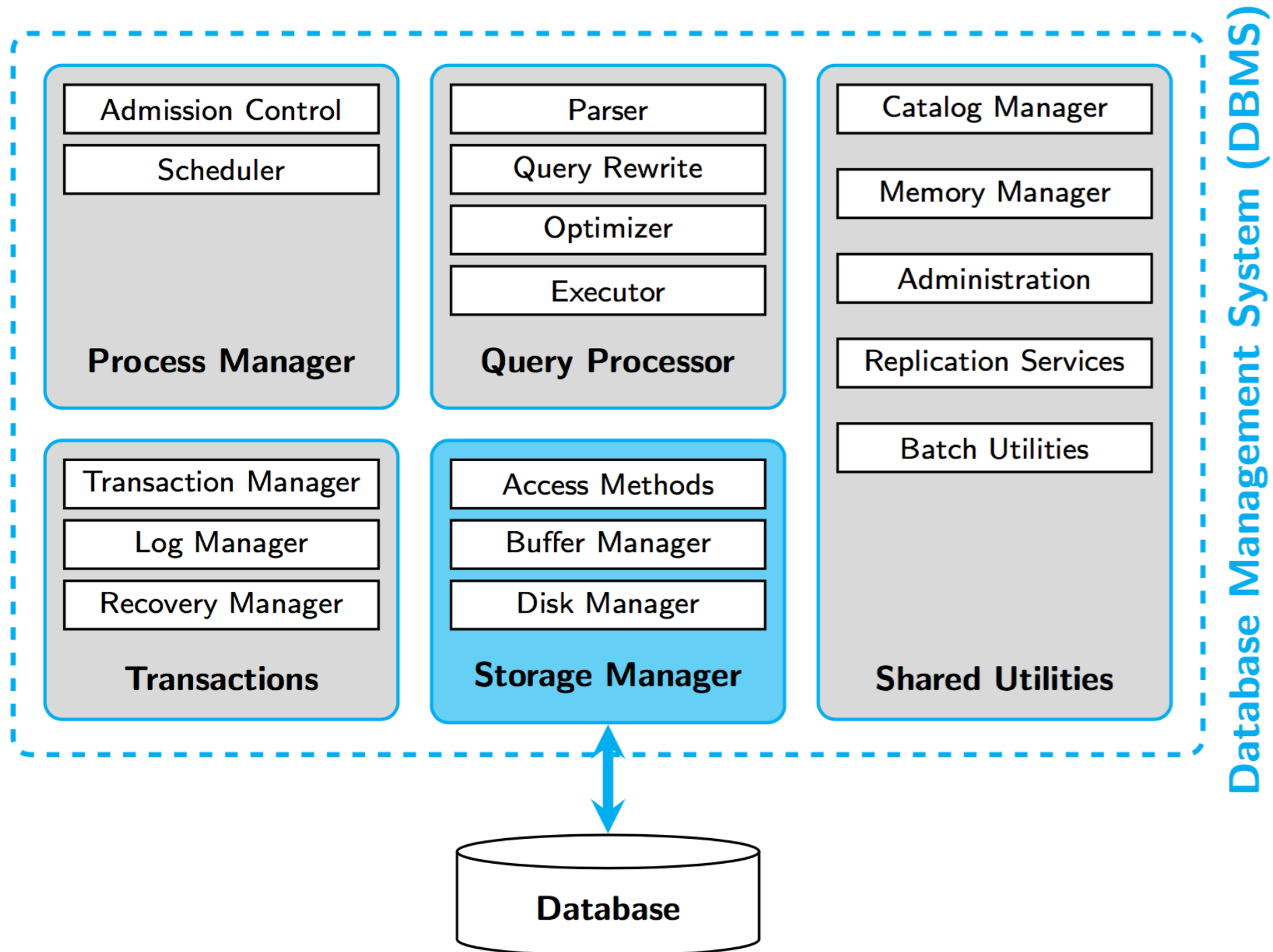
Paso 4

Ejecución

Uso de “pipelining”

Cada operador retorna tuplas a su operador padre
“as soon as possible”

Storage Manager



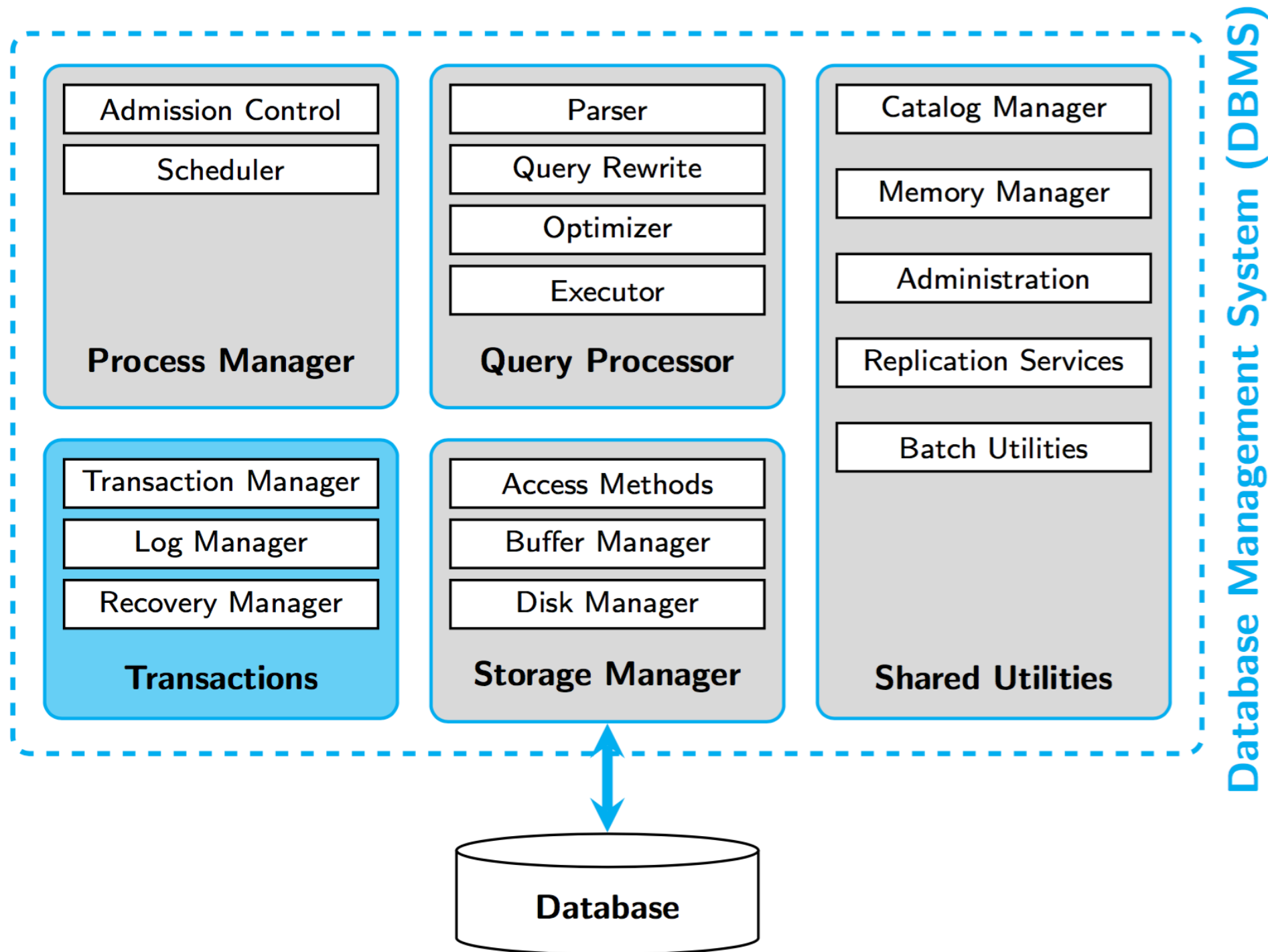
Storage Manager

Disk Manager se encarga del almacenamiento secundario

Buffer Manager se encarga del uso de datos en memoria y optimiza cantidad de I/O al disco duro

Access Methods se encarga de la organización eficiente de los datos

Transactions



Transactions

Componente que asegura las propiedades **ACID**



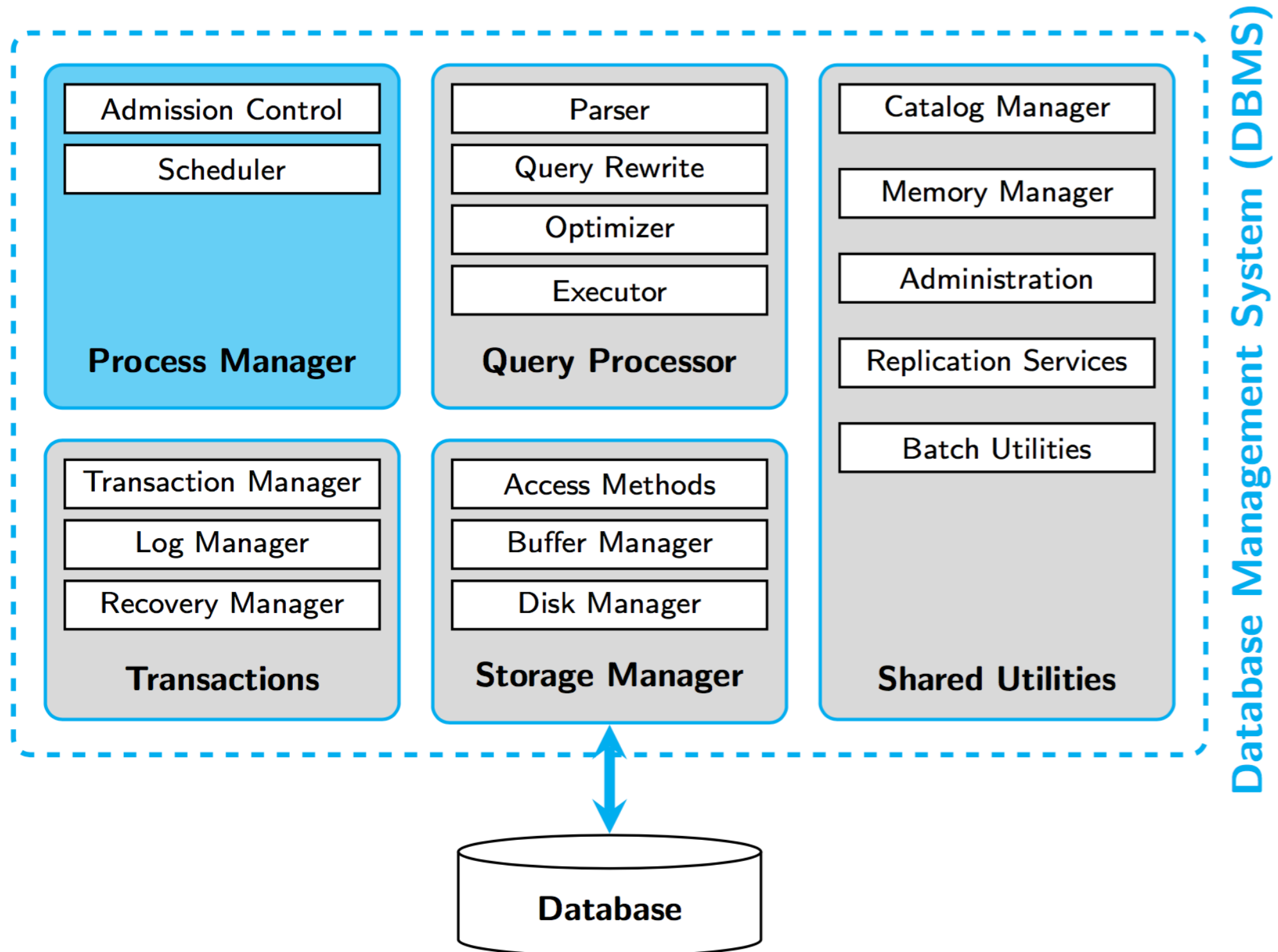
Atomicity
Consistency
Isolation
Durability

Transactions

Transaction Manager se encarga de asegurar Isolation y Consistency

Log y Recovery Manager se encargan de asegurar Atomicity y Durability

Process Manager

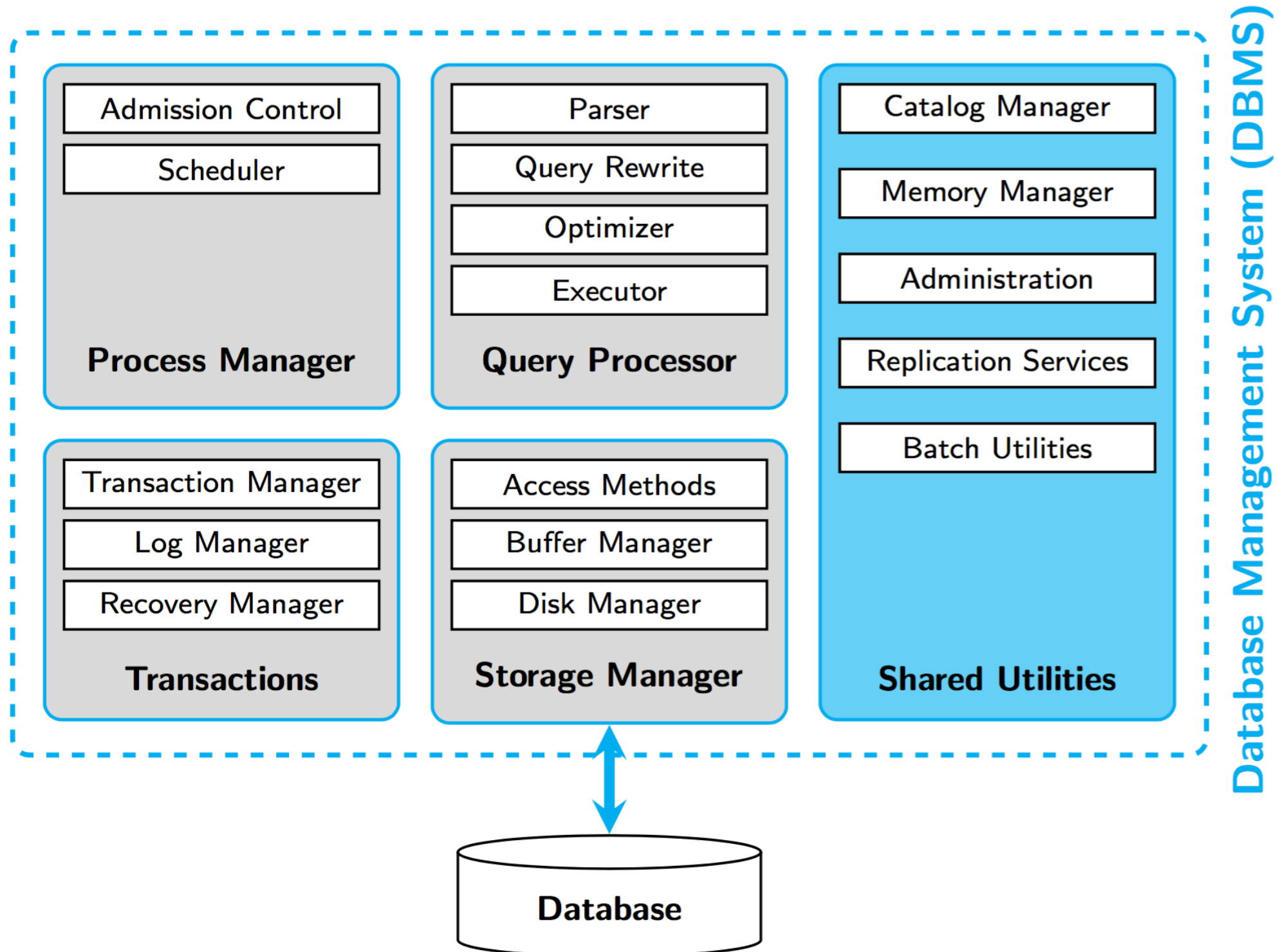


Process Manager

Admisión Control se encarga del manejo de sesiones

Scheduler maneja los procesos y los threads, además de la paralización de consultas

Shared Utilities



Shared Utilities

Catalog Manager maneja los metadatos asociados a los esquemas

Memory Manager Asigna memoria a otras componentes

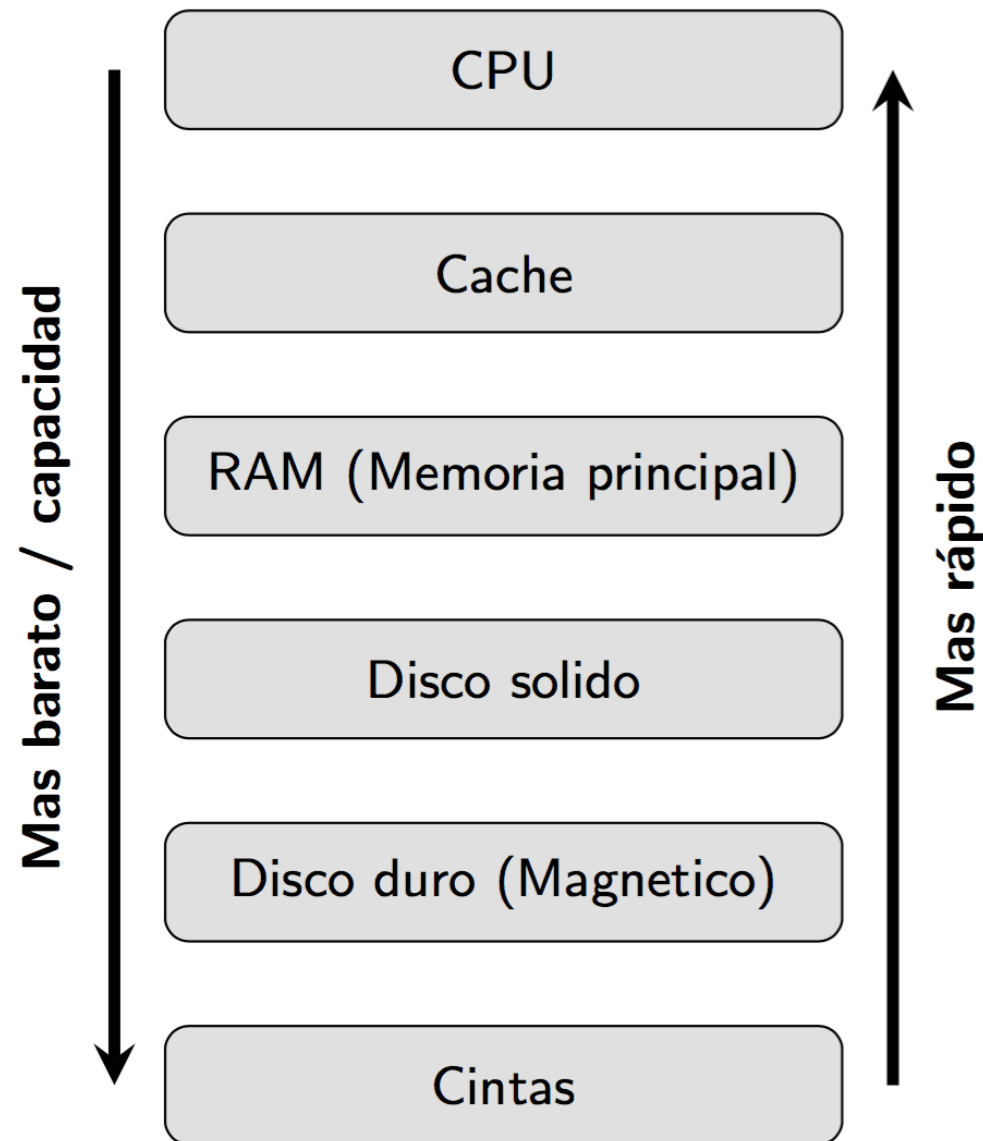
Almacenamiento

Almacenamiento

La base de datos necesita hacer persistente los datos en memoria secundaria

Almacenamiento

Jerarquía de Memoria

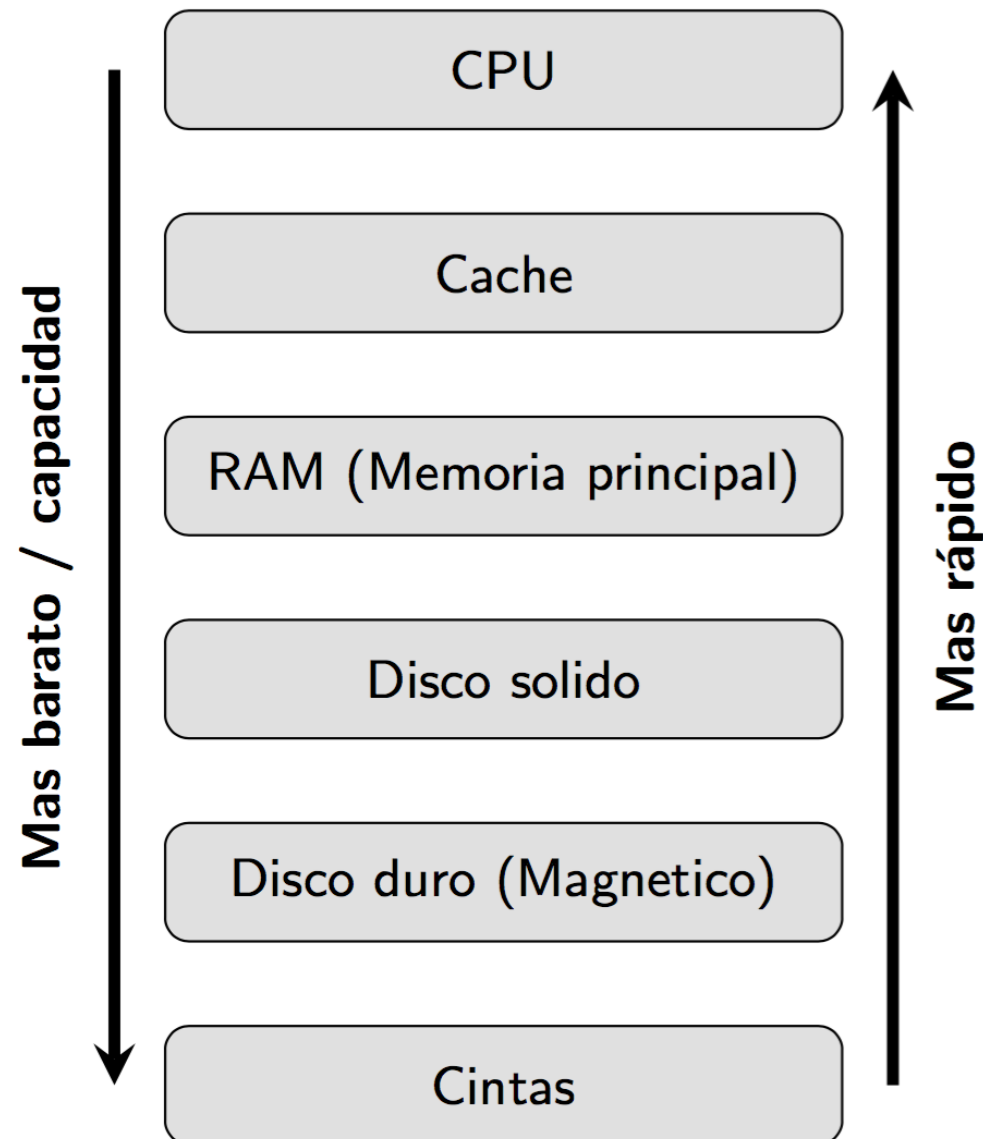


Un acceso a disco es mucho mas lento que acceder a RAM...

... así que hay que minimizarlos!

Almacenamiento

Jerarquía de Memoria

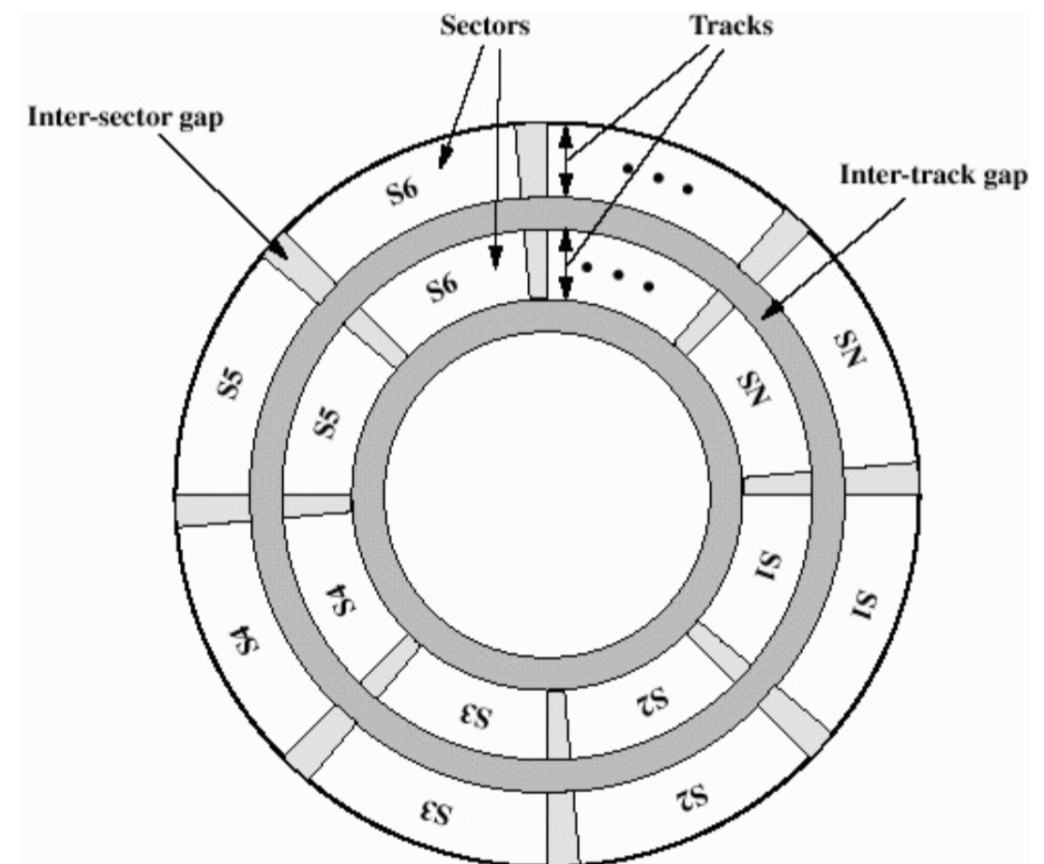
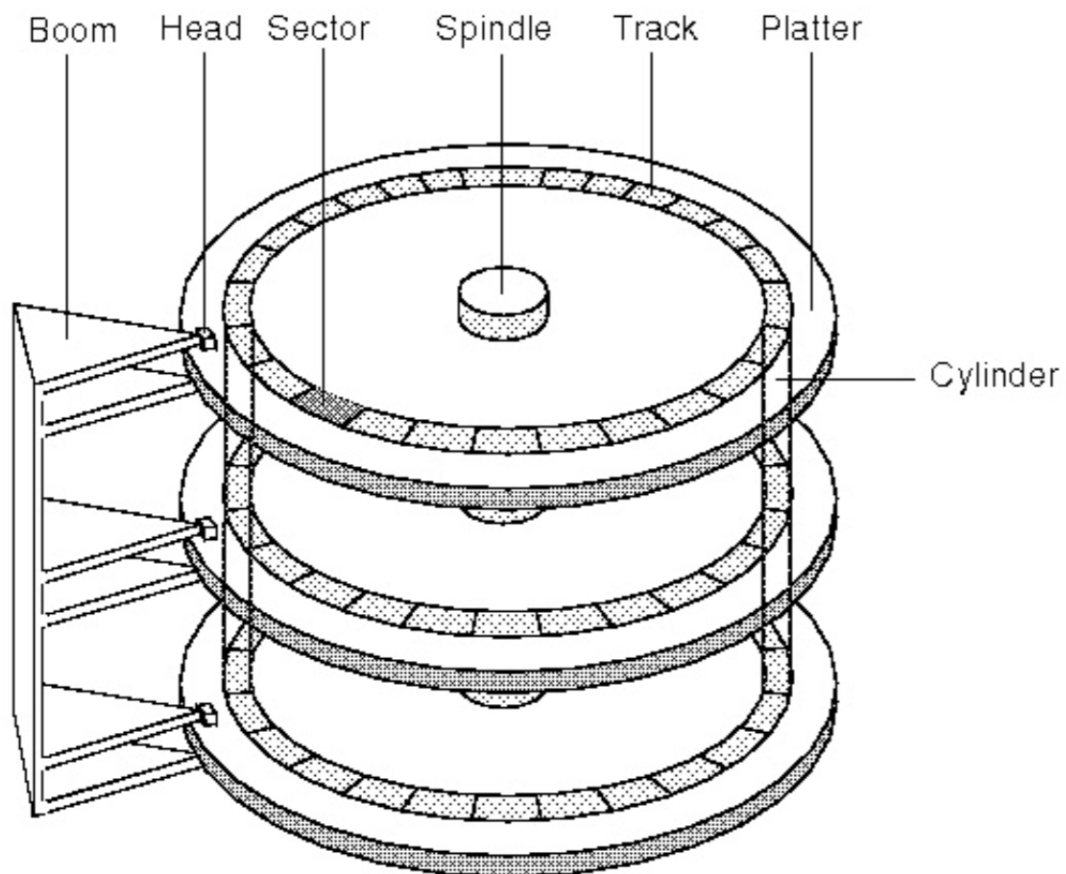


Nivel	Velocidad	Capacidad
Registros CPU	~1 ns	~KB
Caché L1/L2/L3	~5–30 ns	~MB
RAM	~100 ns	~GB–TB
SSD	~100 μ s	~TB
HDD	~10 ms	~TB
Cinta	~segundos	~PB

Disco Duro HDD

Partes importantes

- Sector
- Track
- Gap
- Plato
- Cabeza Lectora



Disco Duro HDD

Ejemplo de disco duro:

Superficies = 16 (8 platos dobles)

Tracks = 2^{16} = 65.536 tracks por superficie

Sectores = 2^8 = 256 sectores por track

|Sector| = 2^{12} = 4.096 bytes por sector

Superficies × **Tracks** × **Sectores** × **|Sector|** ~ 1 TB

Disco Duro

Sector: unidad física mínima de almacenamiento para el Disco

Página: unidad lógica mínima de almacenamiento para el Sistema de Archivos

Los DBMS piensan en Páginas, no en Sectores

Disco Duro y DBMS

Los records de las bases de datos se almacenan en **páginas** de disco.

A medida que se hace necesario, las páginas son traídas a memoria

Demora en Búsqueda

En general, lo más lento del DBMS es ir a buscar los datos a disco, por lo que queremos minimizar el número de I/O

$$\begin{array}{rcl} & \text{Tiempo de Búsqueda} & \\ & + & \\ \textbf{Tiempo Total} = & \text{Delay Rotacional} & \\ & + & \\ & \text{Velocidad de Transferencia} & \end{array}$$

Demora en Búsqueda

Tiempo de Búsqueda: tiempo en posicionar la cabeza en el track

Retraso Rotacional: tiempo en rotar el disco hasta el primer sector que queremos leer

Velocidad de Transferencia: tiempo en leer los sectores

Demora en Búsqueda

Ejemplo de disco duro (2008):

Rotación = 7200 rpm (8.33 ms por vuelta)

Movimiento Brazo = 1 ms para empezar + 1 ms
cada 4000 tracks (65.536 tracks)

Transferencia = 0.13 ms por página

Demora en Búsqueda

	Min	Max	Promedio
Seek	0 ms	17.38 ms	6.46 ms
Rotational	0 ms	8.33 ms	4.17 ms
Transfer	0.13 ms	0.13 ms	0.13 ms
Total	0.13 ms	25.84 ms	10.76 ms

¿Qué pasa si queremos buscar en 1000 páginas distintas?

Demora en Búsqueda

Acceso a 1 página aleatoria:

$$\text{Seek } (\sim 6\text{ms}) + \text{Rotación } (4\text{ms}) + \text{Transfer } (0,13\text{ms}) = 11\text{ms}$$

Acceso a 1000 páginas aleatorias:

$$1000 * 11\text{ms} = 11 \text{ segundos}$$

Acceso a 1000 páginas secuenciales:

$$1 \text{ Seek } (\sim 6\text{ms}) + 1 \text{ Rotación } (4\text{ms}) + 1000 \text{ Transfer } (0,11 \text{ seg})$$

Disco Duro

En general, tiempo de escritura > tiempo de búsqueda

Espacio del Disco >> Espacio en RAM

Tiempo de acceso a Disco >> Tiempo de acceso a RAM

Disco de Estado Sólido (SSD)

No hay costo ni de Seek ni Rotación

Acceso aleatorio toma ~ 0.1 ms (vs 13 en HDD)

Aún así es 1000 veces más lento que una RAM

Pero 1000 accesos secuenciales son solo $\sim 2-4$ veces mas rápidos que 1000 accesos aleatorios.

Nube?

Discos como volúmenes de red, replicados
(pero algo mas lentos)

Pero por mucha realización, el acceso a disco sigue
siendo >> que el acceso a RAM

Todo lo que discutimos acá es válido en la nube



¿Y qué tiene que ver esto con
Bases de Datos?

Cómo se guarda una tabla

Supongamos una tabla $T(a \text{ int}, b \text{ text})$ con 9 tuplas

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

Ojo: las tuplas no necesariamente son adyacentes en el disco duro

Costo I/O (Input/Output)

¡El costo más grande de las bases de datos es traer las páginas del disco duro a memoria RAM!

Queremos responder las consultas haciendo la menor cantidad de lecturas al disco duro

Llamamos costo de I/O al número de páginas llevadas a memoria para responder una consulta

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

¿Cuál es el costo en I/O de hacer la siguiente consulta?

SELECT * FROM T

El costo es 3, porque debo leer las 3 páginas

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

¿Cuál es el costo en I/O de hacer la siguiente consulta?

SELECT T.a FROM T

El costo nuevamente es 3, porque debo leer las 3 páginas

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

¿Cuál es el costo en I/O de hacer la siguiente consulta?

```
SELECT * FROM T WHERE T.a = 4
```

El costo nuevamente es 3, porque debo leer las 3 páginas

Bases de datos Columnares

(Paréntesis)

Los DBMS estándar intentan guardar en páginas adyacentes las filas de una misma tabla (en orden)

Algunos DBMS optan por guardar en páginas adyacentes toda una columna

Bases de datos Columnares

(Paréntesis)

Supongamos la tabla $R(a \text{ text}, b \text{ text})$ con 6 tuplas

Disco Duro

R.a1	R.a2	R.a3	R.a4	R.a5	R.a6	R.b1	R.b2	R.b3	R.b4	R.b5	R.b6
Página 1				Página 2				Página 3			

¿Cuál es la ventaja?

Costo de una consulta

Disco Duro

Tupla 1	Tupla 2	Tupla 3	Tupla 4	Tupla 5	Tupla 6	Tupla 7	Tupla 8	Tupla 9
Página 1			Página 2			Página 3		

¿Cuál es el costo en I/O de hacer la siguientes consultas?

```
SELECT * FROM T WHERE T.a = 4  
SELECT * FROM T WHERE T.a >= 4
```

¿Podemos hacer esto mejor?

Si! Hoy: buffer, cache hits.
Próxima Clase: índices.

El enemigo entonces: Costo I/O

Culpable de gran mayoría de los malos tiempos

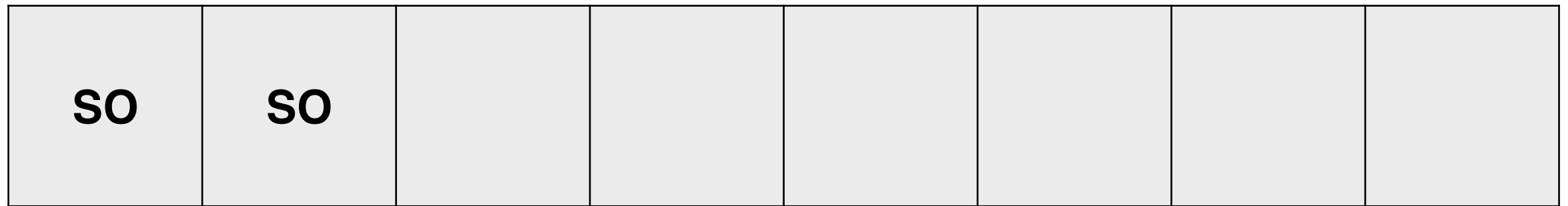
Vamos a intentar minimizarlo a toda costa.

Todo lo que discutimos acá es válido en la nube
(Aunque en la nube hay otras soluciones disponibles)

Buffer Manager

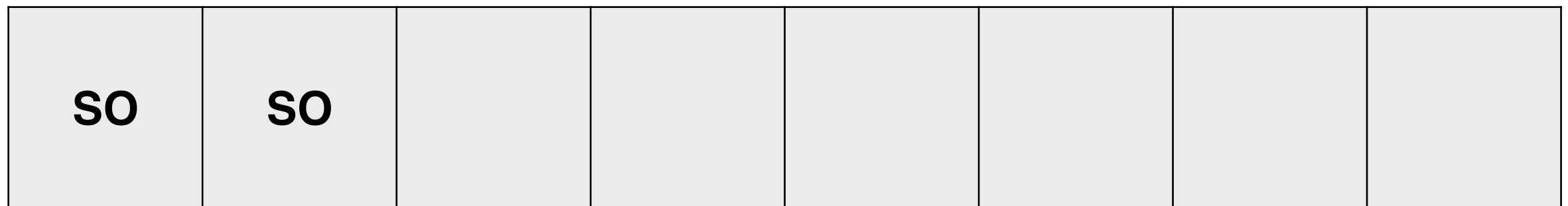
Disco Duro y RAM

RAM

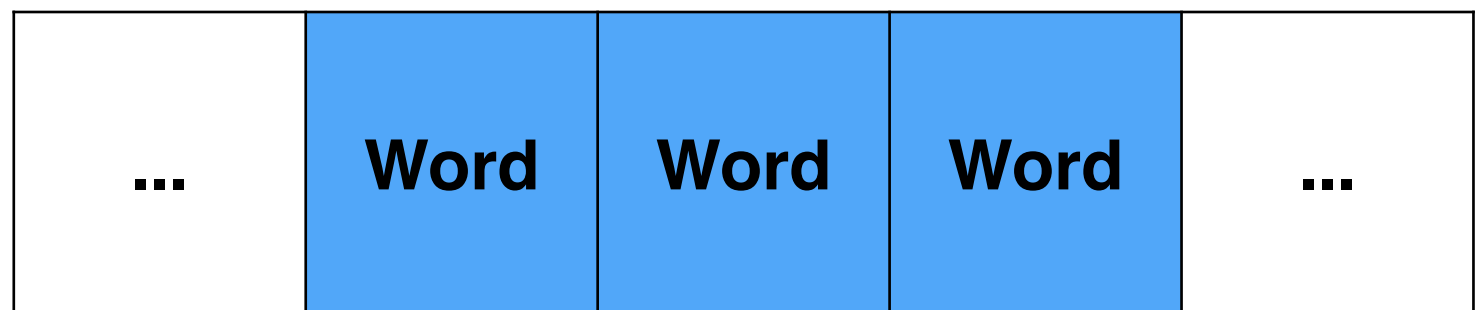


Disco Duro y RAM

RAM

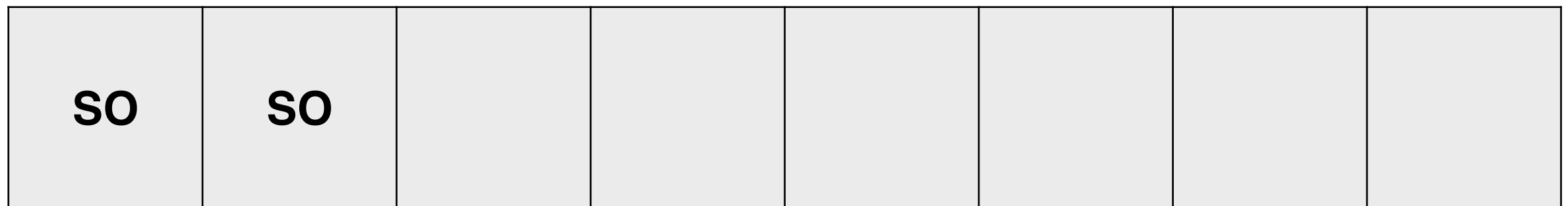


Disco Duro

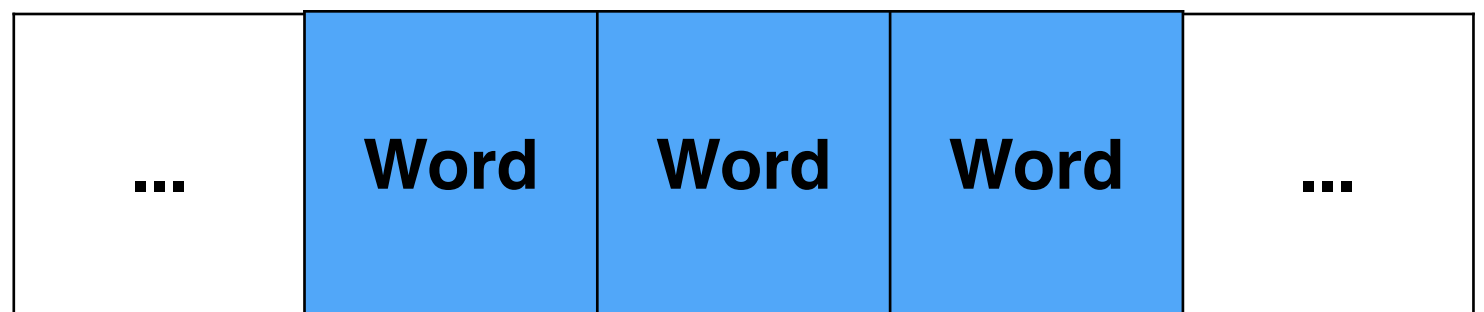


Disco Duro y RAM

RAM

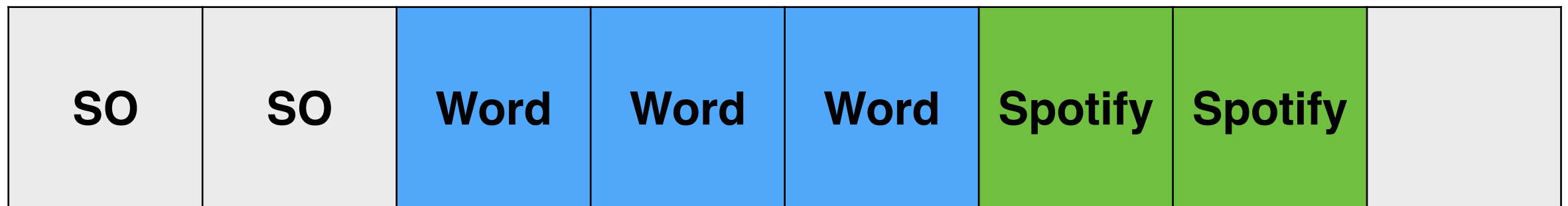


Disco Duro



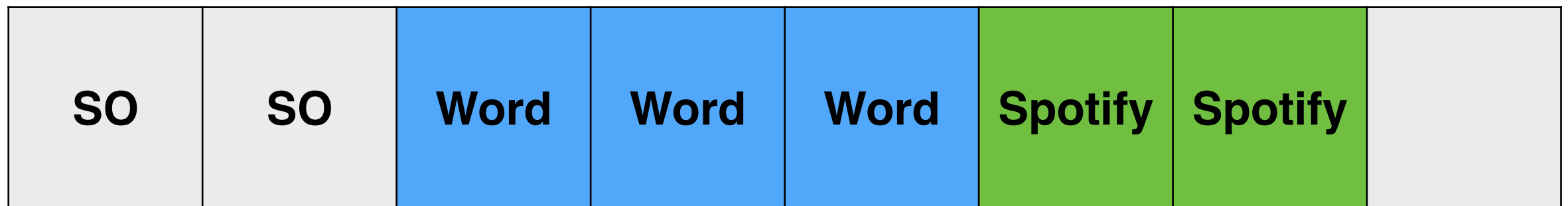
Disco Duro y RAM

RAM



Disco Duro y RAM

RAM

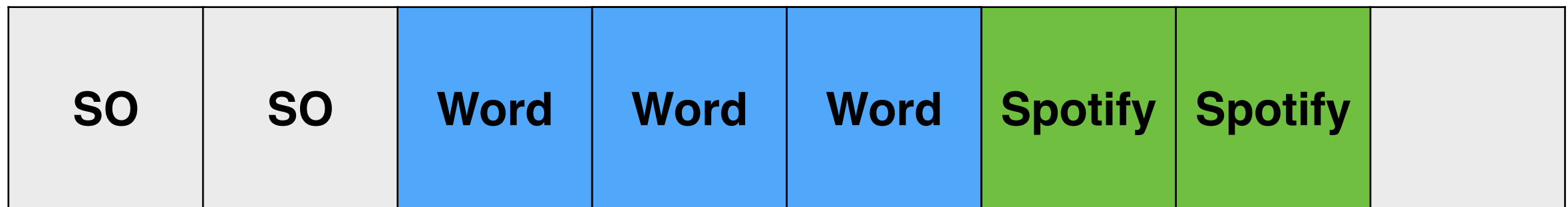


Disco Duro



Disco Duro y RAM

RAM



Disco Duro (espacio de swap)



Disco Duro

Disco Duro y RAM

RAM



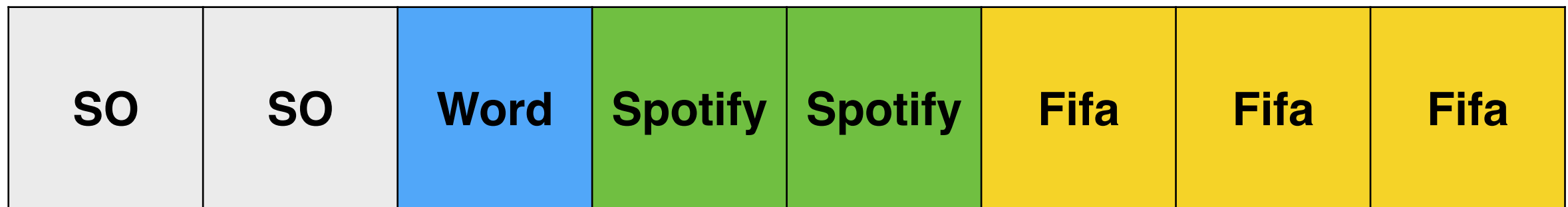
Disco Duro (espacio de swap)



Disco Duro

Disco Duro y RAM

RAM



Disco Duro (espacio de swap)

Disco Duro



Disco Duro y RAM

RAM



Disco Duro (espacio de swap)



Buffer Pool

O a veces conocido como “el cache” (de RAM).

Buffer pool es un arreglo de frames.

Cada frame puede contener una página.

Tamaño del Buffer Pool: B frames (B páginas en RAM)

Metadatos por frame/página:

- **Pin count.** ¿Cuántas operaciones usan esta página?
- **Dirty bit.** Cuando se modifica la página en RAM

Flujo del Buffer Manager

Consulta por una página P.

P en buffer pool -> Buffer hit (no hay costo I/O)

P no en buffer pool -> Traerla a RAM

Buffer pool lleno? -> elegir página para borrar (víctima)

Victima con dirty bit? -> escribirla en disco!

Traer P a RAM

Pinned frame

Si un frame tiene $\text{pin count} > 0$, no se puede borrar

Cuando proceso deja de usarla, $\text{pin count} -= 1$

A quién borro entonces?

Tres políticas para encontrar víctimas.

- LRU (Least Recently Used)
- Clock
- MRU (Most Recently Used)

A quién borro entonces?

LRU (Least Recently Used) - la política más común

Desalojar la página que **no se ha usado hace más tiempo**

Intuición: Si no la uso hace rato, no la necesito

Implementación: Con timestamps de uso por frame

A quién borro entonces?

Clock - LRU mas barato

Cada frame tiene un bit, comienza en 0

Al acceder a una página, se pone un 1

Cuando elijo víctima, recorro el pool con puntero:

- si frame apuntada es 0 -> esa es mi víctima
- Si frame apuntada es 1 -> bit a 0, avanzo

A quién borro entonces?

MRU (Most Recently Used) - ¿ahh?

Desalojar la página usada más recientemente

Anomalía de LRU con sequential scans

Buffer con 3 frames

Tabla T con páginas P1–P5

Consulta **SELECT * FROM T**

Política **LRU**

- I. Cargar P1, P2, P3 → buffer lleno
- II. Necesitamos P4 → desalojar P1 (LRU) → cargar P4
- III. Necesitamos P5 → desalojar P2 (LRU) → cargar P5
- IV. Si hacemos otro scan → P1 ya no está → ****0 hits****

LRU desaloja las páginas que vamos a necesitar pronto!

Buffer con 3 frames

Tabla T con páginas P1–P5

Consulta **SELECT * FROM T**

Política **MRU**

- I. Cargar P1, P2, P3 → buffer lleno
- II. Necesitamos P4 → desalojar P3 (MRU) → cargar P4
- III. Necesitamos P5 → desalojar P4 (MRU) → cargar P5
- IV. P1 y P2 siguen en buffer! Las podemos reutilizar.

A quién borro entonces?

Tres políticas para encontrar víctimas.

- LRU (Least Recently Used)
- Clock
- MRU (Most Recently Used)

No es tan sencillo.

La política cambia según el tipo de operación

Entonces, Buffer Pool

Un buen análisis toma en cuenta un Buffer de tamaño B

Y puede reducir drásticamente el costo I/O en la práctica

PostgreSQL usa un buffer pool compartido llamado `shared_buffers` + el page cache del Sistema Operativo